

LLM Full Fine-Tuning

Technical Project Report

Full supervised fine-tuning of Qwen2.5-0.5B for context-grounded QA and explicit abstention

Validated runtime	Configuration
Compute	NVIDIA A30 24 GB SLURM GPU node g-0
Software	Python 3.11.11 PyTorch 2.11.0+cu128 Transformers 5.17.0
Base model	Qwen/Qwen2.5-0.5B
Task	SQuAD v2 context-grounded QA with answer abstention
Serving	FastAPI backend + browser frontend CUDA inference
Final test	5,940 examples Task F1 0.7580 Task EM 0.7271

Scope note: this is a hands-on end-to-end portfolio project designed to demonstrate production-oriented ML/LLM engineering practices. It is not presented as long-term commercial production experience.

September 2026

1. Executive summary

This project implements a complete full supervised fine-tuning workflow for Qwen2.5-0.5B on a SQuAD v2-style context-grounded question-answering task. The engineering objective is broader than producing a trained checkpoint: the work includes deterministic data preparation, tokenization, hyperparameter search, full-parameter training, checkpoint management, validation-based model comparison, frozen test evaluation, qualitative error analysis, API serving, a browser frontend, lightweight tests, logging, and cluster execution on an NVIDIA A30.

The final model learns two behaviors that the base model does not reliably exhibit: extract a concise answer from the provided context, and explicitly abstain with “I don't know.” when the answer is unsupported. On the frozen 5,940-example test set, final v3 reaches task F1 0.7580 and task EM 0.7271; the unfine-tuned base model reaches task F1 0.1070 and task EM 0.0042.

What was achieved

- A reproducible training pipeline with tracked configuration, deterministic dataset selection, tokenization, full-parameter optimization, validation-loss checkpoint selection, and MLflow logging.
- Three comparable training-data variants evaluated on the same validation split, with the final v3 artifact preserving a deliberate answer/abstention trade-off.
- A frozen base-vs-fine-tuned final-test evaluation plus qualitative error categories and example-level analysis.
- A working FastAPI service, Swagger contract, CUDA-backed inference path, static browser UI, SSH-tunneled remote execution, unit tests, and latency logging.
- A documented reproducibility boundary: the pipeline/configuration is reproducible, but CUDA/bfloat16 training is not claimed to be bitwise deterministic.

2. Problem, objective, and scope

The task is extractive-style question answering with an abstention option. Each request provides a context and a question. The model must answer only from the context; if the context does not support the answer, it must return exactly “I don't know.” This makes the project useful for demonstrating supervised instruction tuning, selective answering, evaluation trade-offs, and deployment discipline in a compact setting.

The project is intentionally not framed as a general-purpose assistant. Its measured claims are limited to the SQuAD v2-derived dataset and the implemented evaluation protocol.

3. Dataset construction and preprocessing

The final v3 training variant contains 30,000 processed examples with a 60/40 answerable-unanswerable mix. Validation and test are held unchanged across the compared variants, preserving comparability.

Dataset item	Value
Processed train total	30,000
Train answerable	18,000
Train unanswerable	12,000 (40%)
Validation	5,933 (unchanged across variants)
Test	5,940 (unchanged across variants)
Selection seed	42

Tokenization and supervision

Qwen chat-template examples are constructed from a system message, a user message containing Context and Question sections, and the assistant answer. Only assistant response tokens participate in the language-model loss; system/user content and the structural assistant prefix are masked with label -100.

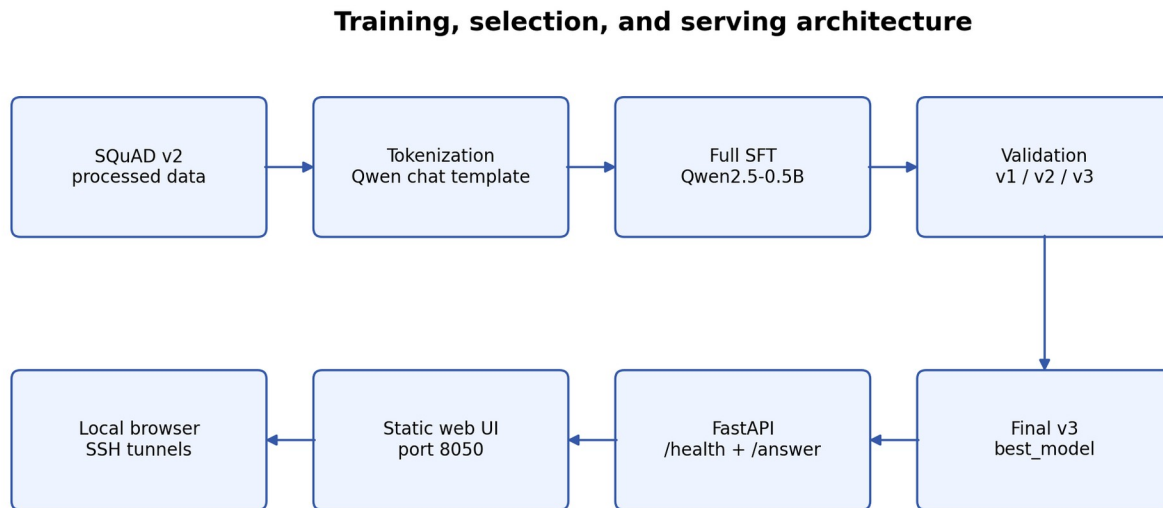
```
System: Answer the question using only the provided context.  
Return only the answer supported by the context.  
If the context does not contain the answer, respond exactly: I don't know.
```

The tokenizer applies a maximum sequence length of 1024 tokens. Three v3 training examples exceed this threshold and are removed, reducing the tokenized training set from 30,000 to 29,997. The preprocessing script also asserts that no retained sample has zero supervised assistant tokens.

Variant design

v3 is an intermediate 60/40 mixture. It retains the 18,000 answerable examples and 10,000 unanswerable examples from v1, then adds 2,000 unanswerable examples using the same deterministic selection procedure used for the more abstention-heavy v2 construction. This creates an explicit distributional lever for studying false answers versus false abstentions.

4. System architecture



Canonical runtime: DANTZIG/UMH NVIDIA A30 GPU | Python 3.11.11 | CUDA-backed PyTorch

Figure 1. End-to-end training, model-selection, and serving architecture.

Runtime request path

- The local browser sends context and question text to the FastAPI POST /answer endpoint.
- FastAPI rebuilds the same system/user chat messages used during training and final evaluation.
- The Qwen chat template is applied with `add_generation_prompt=True`.
- The final v3 best_model runs on CUDA in bfloat16 inference mode.
- Generation is deterministic: `do_sample=False`, `repetition_penalty=1.0`, `max_new_tokens=64`.
- The API returns the generated answer and the number of input tokens; inputs above 1024 tokens are rejected with HTTP 422.
- The static frontend displays the answer and API/GPU status.

5. Full fine-tuning methodology

The project performs full-parameter supervised fine-tuning. No LoRA or QLoRA adapters are used: the complete Qwen2.5-0.5B model is optimized.

Training parameter	Final value
Model	Qwen/Qwen2.5-0.5B
Epochs	3
Per-device batch size	4
Gradient accumulation	4
Effective batch size	16
Precision	bf16 autocast
Learning rate	2.5789950779174585e-05
Weight decay	0.02433937943676302
Scheduler	linear
Warmup ratio	0.018118910790805944
Max grad norm	1.0
Seed	42

Optimization details

The custom training loop weights loss accumulation by supervised assistant-token count. Before each optimizer update, accumulated gradients are divided by the accumulated supervised-token count, then gradient norm is clipped. Validation loss is computed as mean cross-entropy per supervised assistant token.

The best checkpoint is selected by minimum validation loss and saved with model, tokenizer, and the effective training configuration. A separate last_checkpoint retains optimizer state, scheduler state, best-epoch metadata, and RNG state for recovery.

Hyperparameter search

The tracked configuration defines a 12-trial Optuna search with a 600-optimizer-step surrogate horizon. The search spans learning rate, weight decay, warmup ratio, and linear/cosine scheduling. Three candidates are configured for confirmation; the final full run uses the candidate labelled trial_3.

6. Training telemetry and checkpoint selection

Historical stdout logs were intentionally cleaned during project maintenance, so a controlled rerun of the final v3 configuration was executed into a separate checkpoint directory to recover operational telemetry without overwriting the original artifact. The rerun used the same saved configuration and dataset.

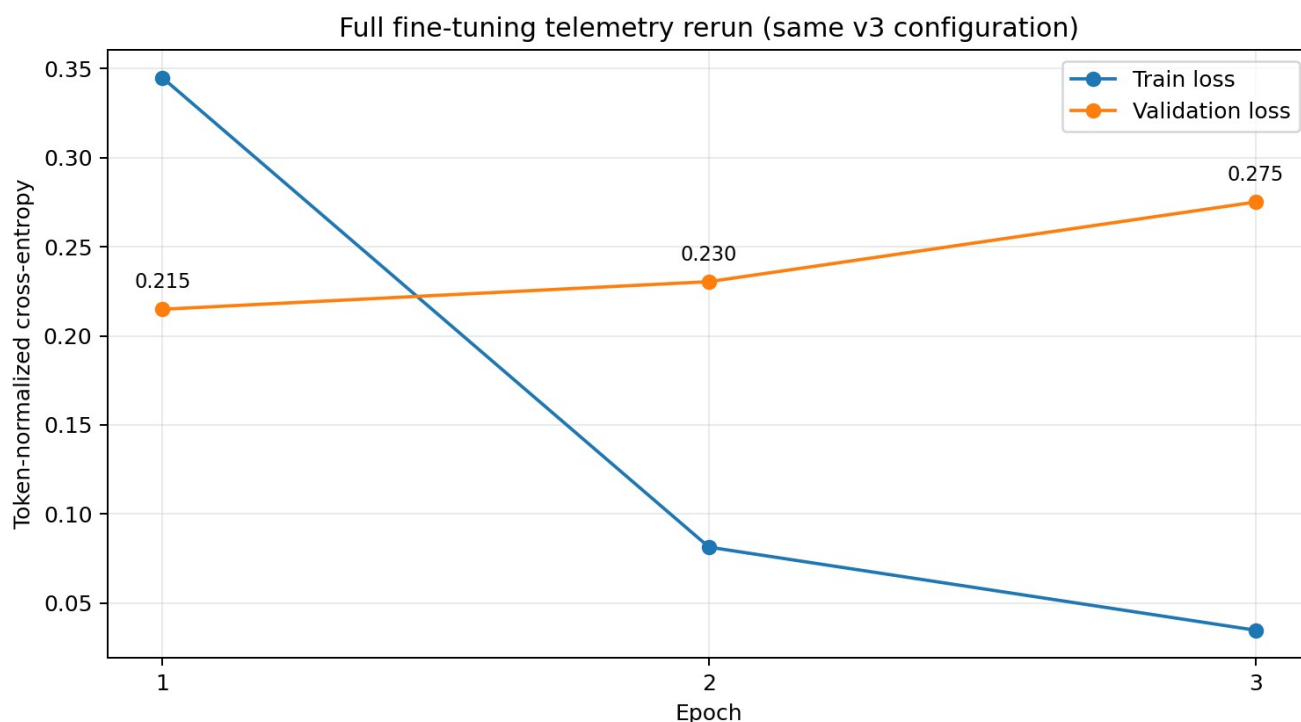


Figure 2. Full fine-tuning telemetry rerun. Validation loss is lowest after epoch 1 while training loss continues to decline.

Telemetry	Result
Best epoch	1
Best validation loss	0.2149047066
Training time	2548.86 s (~42 min 29 s)
Peak allocated VRAM	14.998 GiB
Optimizer steps	5,625
Warmup steps	101

The pattern after epoch 1 is a clear overfitting signal: training loss decreases sharply while validation loss worsens. This validates the decision to keep the epoch-1 `best_model` rather than the final epoch state.

Reproducibility check

The original and rerun `best_model` directories store identical `training_config.yaml` content, but their `model.safetensors` SHA-256 hashes differ. Therefore the project does not claim bitwise deterministic CUDA training. The stronger and defensible claim is reproducibility of data selection, configuration, training procedure, evaluation, and artifact-selection logic.

7. Validation comparison and final artifact

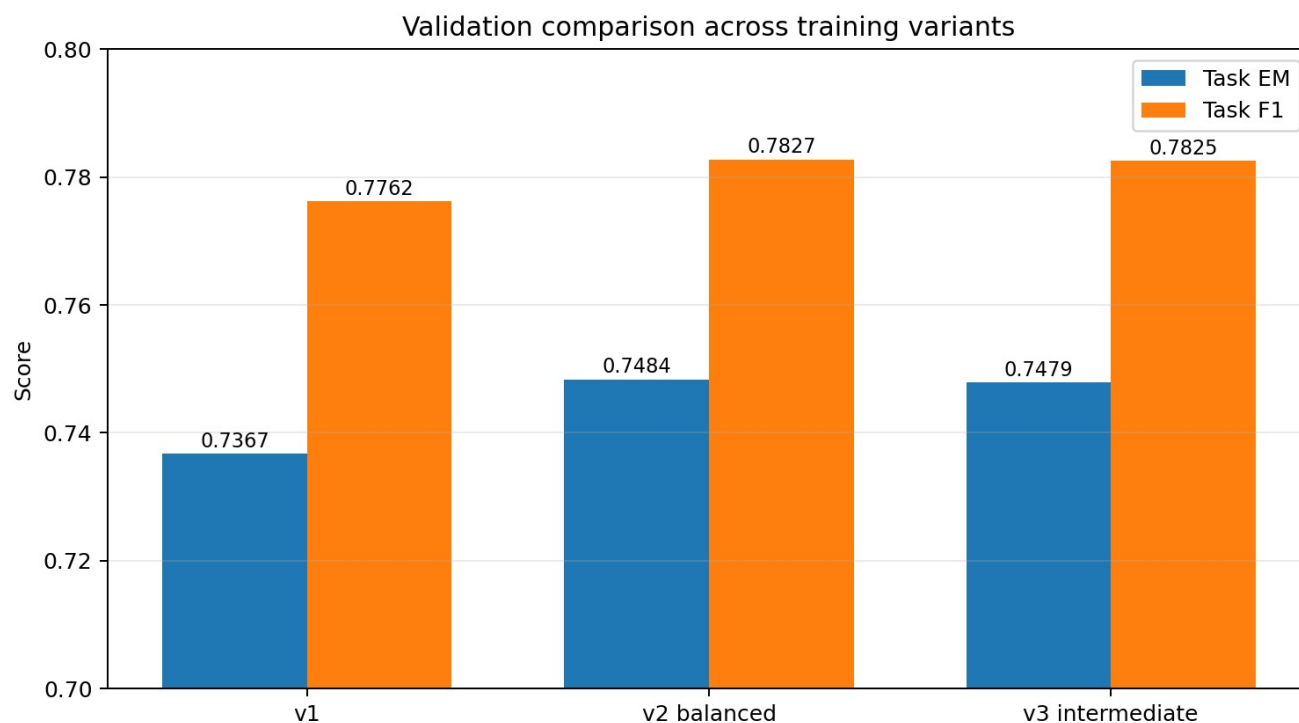


Figure 3. Validation task Exact Match and F1 across v1, v2 balanced, and v3 intermediate.

Variant	Task EM	Task F1	Answerable F1	Unans. abstention	False abstention
v1	0.7367	0.7762	0.7861	0.7664	0.1172
v2 balanced	0.7484	0.7827	0.7128	0.8520	0.2052
v3 intermediate	0.7479	0.7825	0.7354	0.8292	0.1727

Interpretation

v2 and v3 are effectively tied on aggregate task EM/F1, but they occupy different points on the answer/abstention trade-off. v2 is more conservative on unanswerable questions, reaching 85.20% abstention accuracy, while v3 lowers the answerable false-abstention rate from 20.52% to 17.27% and improves answerable F1. The final artifact used by the project is v3 intermediate.

The final-test set is not used to revisit that decision. Test remains a frozen post-selection evaluation.

8. Frozen final-test evaluation

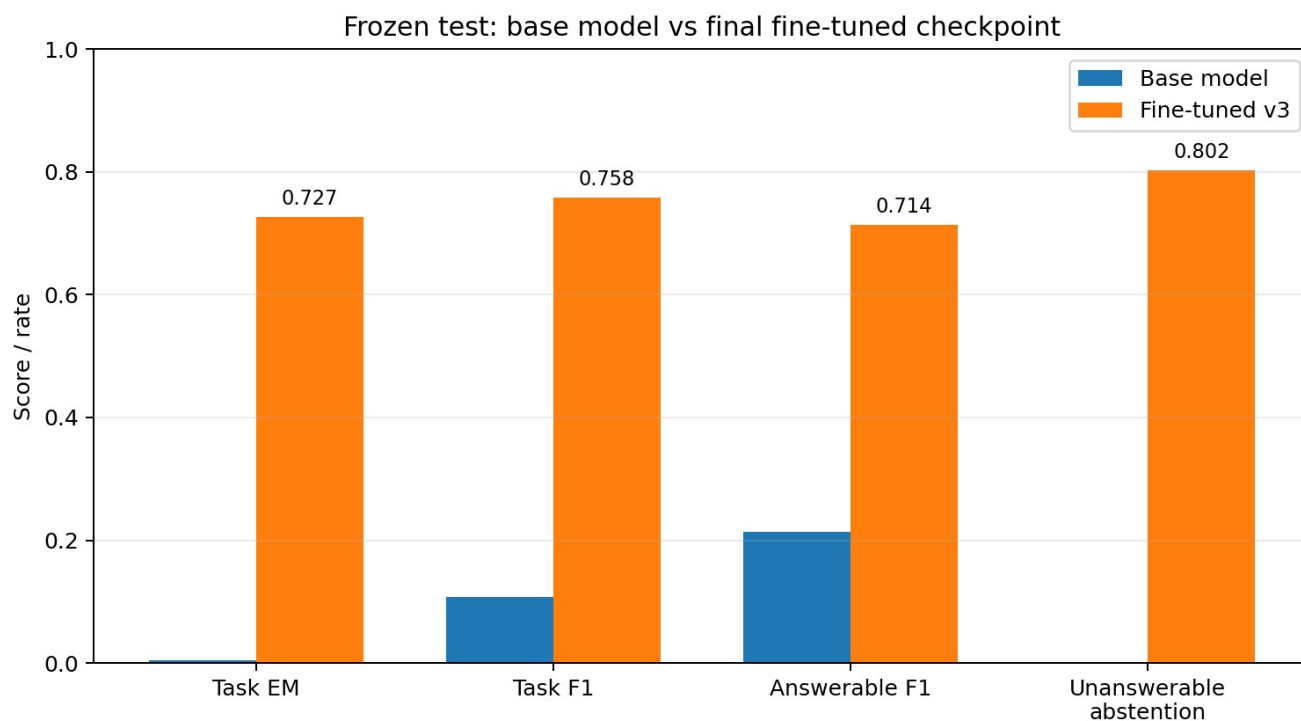


Figure 4. Frozen test comparison between the unfine-tuned base model and final v3.

Metric	Base model	Fine-tuned v3
Task Exact Match	0.0042	0.7271
Task F1	0.1070	0.7580
Answerable F1	0.2136	0.7140
Answerable false-abstention	0.0000	0.1839
Unanswerable abstention accuracy	0.0000	0.8020
Unanswerable false-answer rate	1.0000	0.1980

The base model practically never abstains: its unanswerable abstention accuracy is 0% and false-answer rate is 100%. Fine-tuning changes this behavior materially while also improving answer extraction. The final model still has meaningful residual error: approximately 19.8% of unanswerable examples receive a false answer, and 18.4% of answerable examples receive a false abstention.

9. Qualitative error analysis

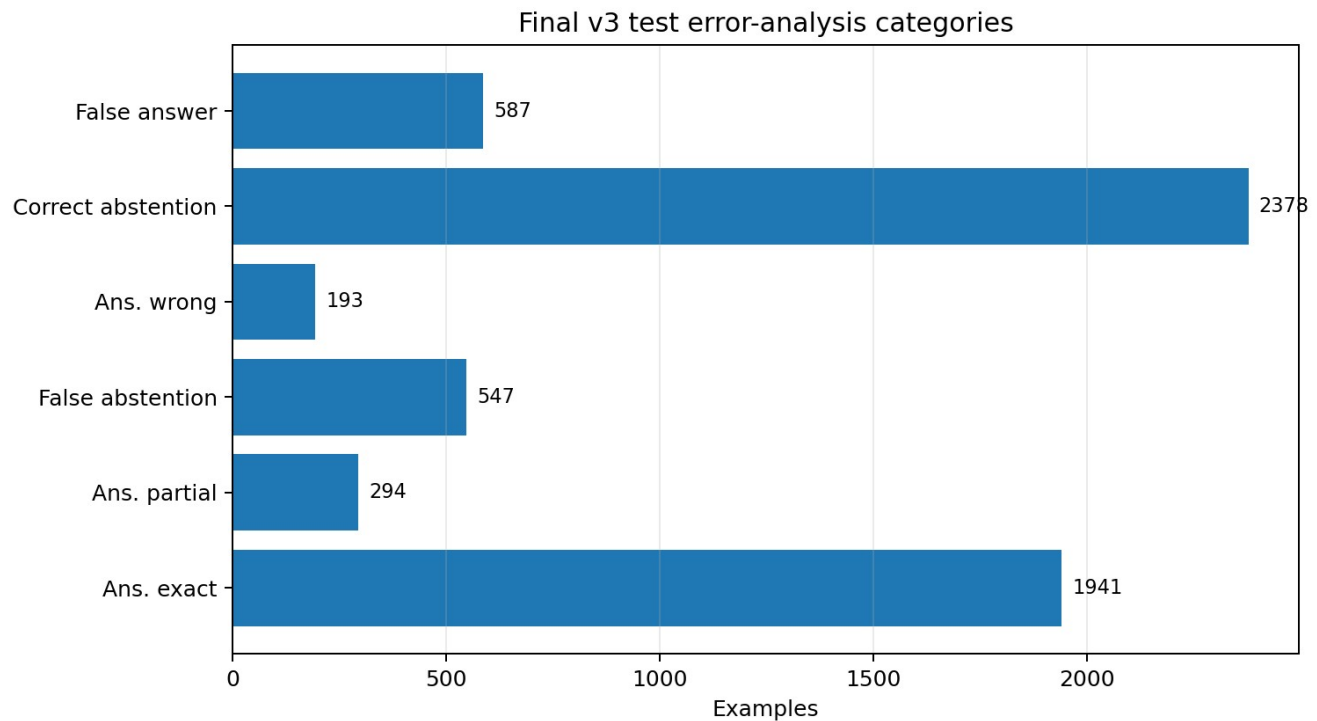


Figure 5. Final v3 test predictions grouped into answerability and error categories.

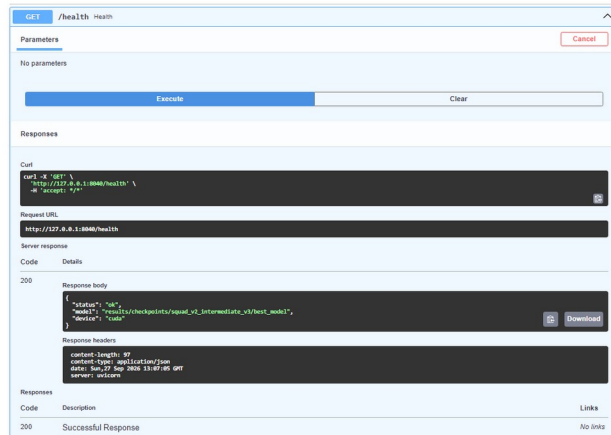
Representative failure modes

- Partial-span differences: “British Gas” versus the gold “British Gas plc” receives partial credit rather than exact match.
- Lexically different but semantically close answers can receive zero token-overlap F1, e.g. “stationary” versus “stagnant”.
- False abstention occurs even when the answer is explicit, such as returning “I don't know.” for “antigen presentation”.
- Entity-selection mistakes occur when several plausible entities are present in the same context, e.g. choosing “WWVA Jamboree” rather than “The Fresno Barn”.
- Some SQuAD unanswerable questions contain strongly related facts in the context. The model may answer that nearby fact (for example “151”) even when the annotated task expects abstention.

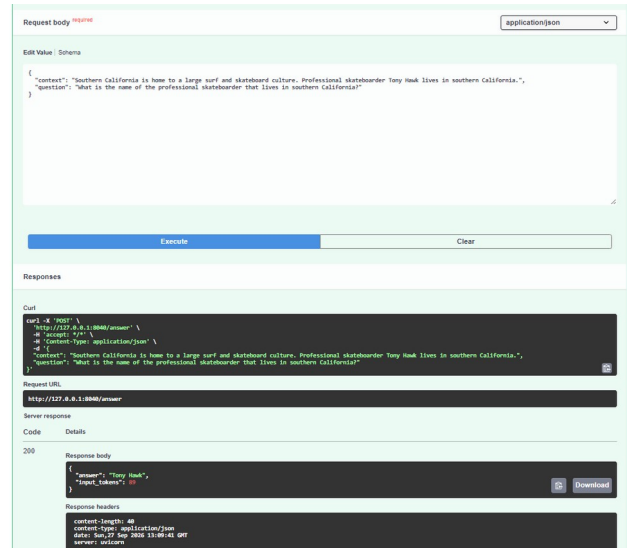
These categories are useful because they separate model limitations from metric artifacts. Exact Match is intentionally strict; qualitative inspection shows when the failure is span formatting, semantic substitution, over-abstention, or genuinely wrong evidence selection.

10. Serving and API behavior

FastAPI initializes the tokenizer and final checkpoint once during application lifespan. The canonical serving path requires CUDA, loads the selected `best_model` in `bfloat16`, enables KV-cache inference, and exposes typed `/health` and `/answer` endpoints.



Health endpoint: HTTP 200 with final model path and CUDA device.



Answerable request: the model returns “Tony Hawk”.

Figure 6. Swagger/OpenAPI evidence for health and a real answerable inference request.

Edit Value | Schema

```

{
  "context": "string",
  "question": "string"
}

```

Execute
Clear

Responses

Curl

```

curl -X 'POST' \
  'http://127.0.0.1:8040/answer' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "context": "string",
    "question": "string"
  }'

```

Request URL

```

http://127.0.0.1:8040/answer

```

Server response

Code	Details
200	Response body <pre> { "answer": "I don't know.", "input_tokens": 55 } </pre> Download Response headers <pre> content-length: 44 content-type: application/json date: Sun, 27 Sep 2026 13:08:21 GMT server: uvicorn </pre>

Responses

Code	Description	Links
200	Successful Response	No links

Figure 7. Swagger unanswerable request: the API returns the exact abstention string “I don't know.”.

Input validation

The backend rejects requests whose fully formatted prompt exceeds 1024 tokens with HTTP 422. Pydantic also enforces non-empty context and question fields.

11. Frontend and remote cluster operation

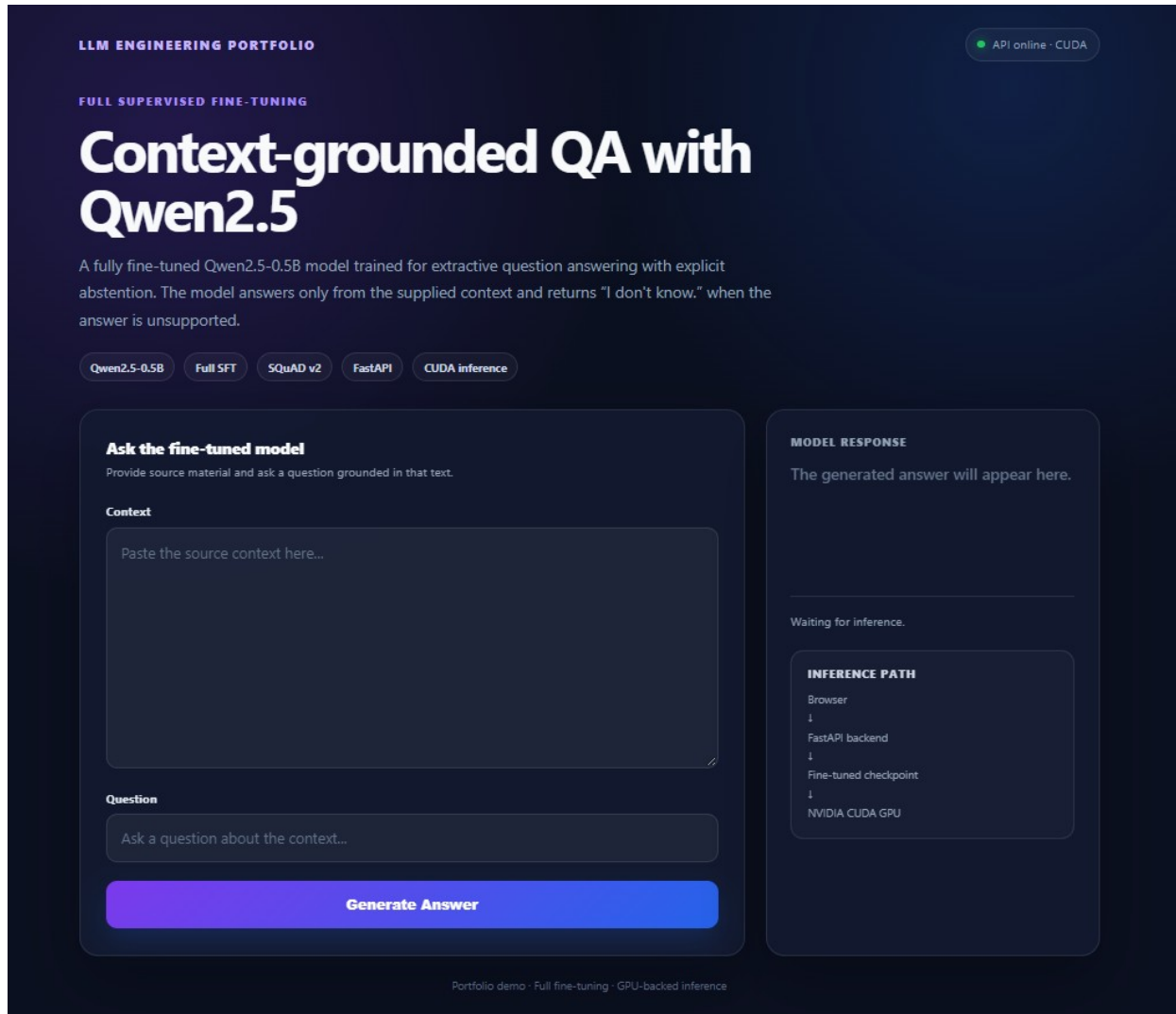


Figure 8. Final browser frontend. The UI exposes model/task scope, API status, CUDA status, context/question inputs, and the inference path.

The frontend is deliberately lightweight static HTML/CSS/JavaScript. It keeps serving concerns separate from model logic: the browser calls FastAPI and displays the returned answer and token count. The interface also checks `/health` so the user can see whether the inference API is online on CUDA.

Validated DANTZIG/UMH workflow

```
sinteractive --partition GPU --qos gpu -w g-0 --cpus-per-task=24 --mem=72G --gres=gpu:1 --time=08:00:00
```

The backend and frontend run on GPU node `g-0` while the local Windows browser remains outside the cluster. Because VS Code Remote SSH is connected to the login node, two internal tunnels were maintained from dantzig to `g-0`, and VS Code forwarded the corresponding ports to the local machine:

```
ssh -N -L 8040:127.0.0.1:8040 g-0
ssh -N -L 8050:127.0.0.1:8050 g-0
```

This operational detail matters: printing a remote `http://127.0.0.1:PORT` URL does not make the service reachable from the local browser without the forwarding path.

12. Testing and observability

Lightweight backend tests

The final lightweight unit suite validates API logic without loading the real checkpoint. Fake tokenizer/model objects are injected into `app.state` so the tests can run quickly on CPU.

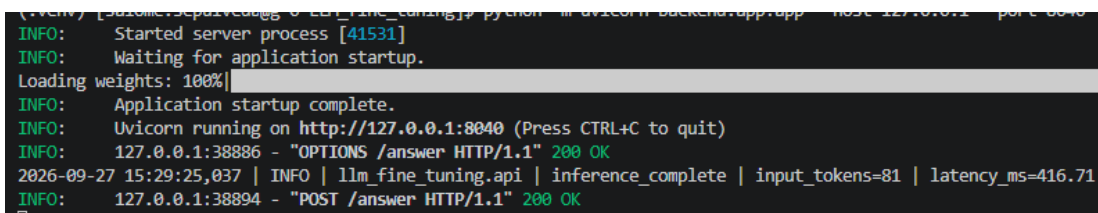
```
test_answer ... ok
test_health ... ok
test_overlength_input ... ok
```

```
Ran 3 tests in 0.092s
OK
```

- Health endpoint contract: status, model path, and declared CUDA device.
- Answer endpoint contract: generated answer and input-token count.
- Overlength input behavior: HTTP 422 when the formatted prompt exceeds 1024 tokens.

Inference logging

Application-level logging records completed inference calls with input-token count and elapsed latency, alongside standard Uvicorn access logs. This provides lightweight runtime observability without claiming a full external monitoring stack.



```
(venv) [satomc@server:~]$ python -m uvicorn backend:app --host 127.0.0.1 --port 8040
INFO: Started server process [41531]
INFO: Waiting for application startup.
Loading weights: 100%|
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8040 (Press CTRL+C to quit)
INFO: 127.0.0.1:38886 - "OPTIONS /answer HTTP/1.1" 200 OK
2026-09-27 15:29:25,037 | INFO | llm_fine_tuning.api | inference_complete | input_tokens=81 | latency_ms=416.71
INFO: 127.0.0.1:38894 - "POST /answer HTTP/1.1" 200 OK
```

Figure 9. Real backend inference log showing `input_tokens=81` and `latency_ms=416.71` for a successful `POST /answer` request.

13. Reproducibility and repository packaging

The project is designed to keep source code, configuration, preprocessing/training/evaluation scripts, tests, SLURM wrappers, result JSON/CSV files, README, and technical report under version control while excluding the large virtual environment and model/checkpoint artifacts from normal Git history.

- pyproject.toml defines the package, supported Python range, and primary dependencies.
- requirements-lock.txt captures the exact validated environment, including PyTorch 2.11.0+cu128, Transformers 5.17.0, FastAPI 0.141.1, and Uvicorn 0.53.0.
- training.yaml defines training, tuning, confirmation, and generation configuration.
- training_config.yaml is persisted with checkpoints so the artifact records the effective data path and configuration.
- Dataset-selection and training seeds are fixed at 42.
- Validation and test result files preserve predictions and aggregate metrics for auditability.
- Model weights are intentionally treated as external large artifacts rather than normal Git content.

Reproducibility boundary

A fresh clone cannot serve the final model until the best_model artifact is restored or regenerated. The project also does not claim bitwise deterministic CUDA training: a controlled rerun used the same configuration but produced different model SHA-256 hashes. This is documented explicitly rather than hidden.

Validated environment

Component	Validated value
Scheduler	SLURM
GPU node	g-0
Partition / QoS	GPU / gpu
GPU	NVIDIA A30 24 GB
Python	3.11.11
PyTorch	2.11.0+cu128
Transformers	5.17.0
FastAPI	0.141.1
Uvicorn	0.53.0

14. Limitations

- The canonical serving workflow requires a CUDA GPU and is not maintained as a 24/7 public service.
- The final checkpoint is approximately 954 MB and is not distributed through ordinary Git history.
- Lightweight backend tests cover API logic but do not yet constitute a full GPU integration or load-testing suite.
- No Docker image, CI/CD pipeline, authentication layer, rate limiting, or concurrency benchmark is currently claimed.
- The final model still produces false answers on some unanswerable inputs and false abstentions on some answerable inputs.
- SQuAD Exact Match/F1 are intentionally strict and can penalize semantically plausible aliases or paraphrases.
- Historical training stdout/stderr was cleaned; operational training telemetry in this report comes from a controlled rerun of the same v3 configuration.
- Bitwise GPU-training reproducibility is not claimed.

15. What would change at production scale

- Dedicated model-serving infrastructure with explicit concurrency, batching, queueing, and GPU admission controls.
- Containerization and CI/CD with reproducible image builds and automated test/evaluation gates.
- Authentication, authorization, quotas, and rate limits around inference endpoints.
- Centralized metrics, traces, structured logs, dashboards, and alerting.
- Load, latency, failure-injection, and security testing under representative traffic.
- Artifact registry/model registry with explicit promotion rules and checkpoint versioning.
- Broader evaluation sets, slice analysis, calibration/abstention tuning, and regression gates before promotion.
- Potential quantization or optimized serving if latency/cost requirements justify the quality trade-off.

16. Conclusion

The project demonstrates a complete full fine-tuning lifecycle rather than an isolated training notebook. It starts with controlled dataset variants, applies Qwen-specific supervised preprocessing, searches and confirms hyperparameters, trains all model parameters on an NVIDIA A30, selects checkpoints using validation loss, compares variants on a fixed validation set, evaluates the final artifact once on a frozen test set, inspects qualitative errors, and exposes the selected model through a working API and browser frontend.

The most useful result is not a single headline metric. The project makes trade-offs visible: more unanswerable training data improves abstention but can increase false abstention; final v3 preserves nearly the same aggregate validation score as v2 while answering more answerable cases; and the final test still exposes both false answers and false abstentions. The engineering workflow preserves those limitations instead of hiding them.

Appendix A. Key commands

Interactive GPU allocation

```
sinteractive --partition GPU --qos gpu -w g-0 --cpus-per-task=24 --mem=72G --gres=gpu:1 --time=08:00:00
```

Backend

```
python -m uvicorn backend.app:app --host 127.0.0.1 --port 8040
```

Frontend

```
.venv/bin/python -m http.server 8050 --bind 127.0.0.1 --directory frontend
```

Lightweight tests

```
python -m unittest tests/test_backend.py -v
```

Appendix B. Main evidence

- Validation v1/v2/v3 comparison JSON with full predictions.
- Frozen final-test base-vs-v3 predictions and aggregate metrics.
- Qualitative analysis and enriched question/context examples.
- Dataset audit for the 60/40 v3 construction.
- Training telemetry rerun with best epoch, validation loss, time, and peak VRAM.
- Swagger health/answerable/unanswerable responses.
- Final browser frontend and CUDA API status.
- Passing backend unit tests.
- Inference latency log.